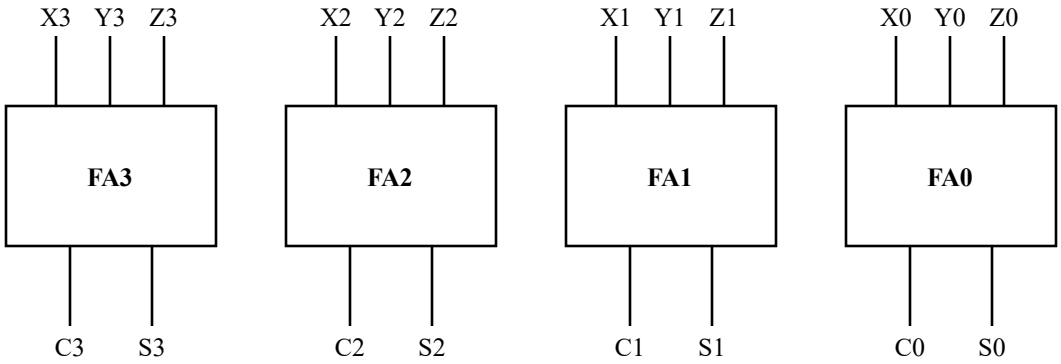
Amrita School of Engineering, Bengaluru	
Department of ECE	
B. Tech. VII Semester (ECE)	
AY 2026-2027 (ODD Semester)	
23ECE336- VLSI TESTING AND TESTABILITY	
Simulation Assignment Report	
Register Number of the Student	
Name of the Student	
Date of Evaluation:	

Design a digital logic system using Verilog HDL and perform simulation to demonstrate the following testing algorithms: fault list reduction, test pattern generation, fault simulation. Analyze and report the simulation results-- CO2, CO3, CO4

Logic Design Name: 4-bit Carry-Save Adder (3:2 compressor) – module `csa_4bit`

The carry-save adder adds three 4-bit operands X, Y and Z and produces a sum vector S and a carry vector C, such that X + Y + Z = S + 2C. It has 12 primary inputs, 8 primary outputs, 20 logic gates, 3 logic levels and no flip-flops (purely combinational).

Logic Diagram:



Carry vector C has weight 2 (shifted left by one bit); result = S + 2C

Fig. 1 – Block diagram: four independent full adders form the 4-bit carry-save adder

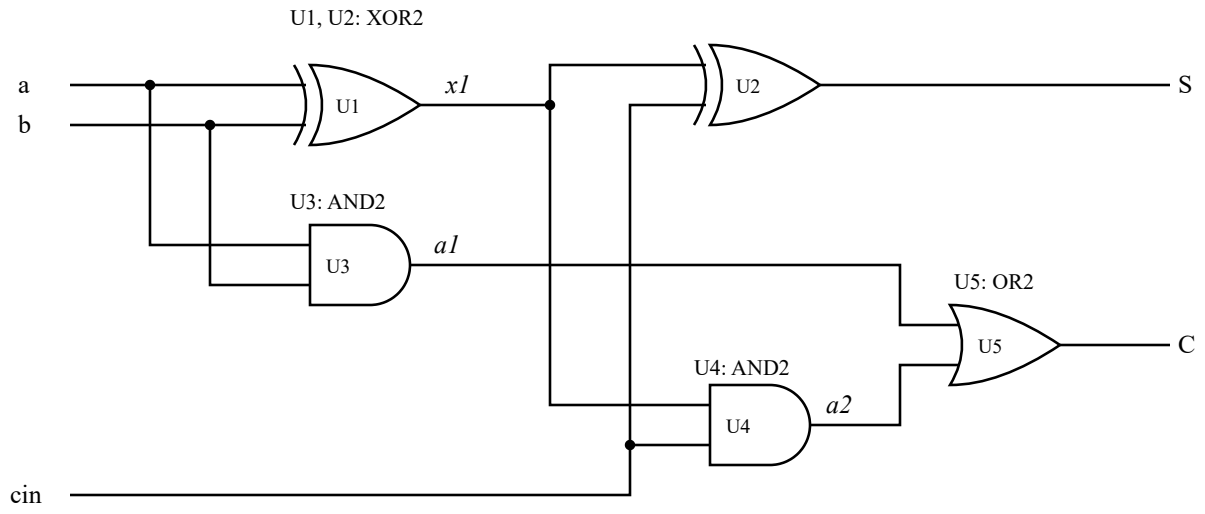


Fig. 2 – Gate-level diagram of one full adder (S = a ⊕ b ⊕ cin, C = a·b + (a ⊕ b)·cin); repeated for bits 0–3

1. Algorithm for Fault list reduction

Aim: To reduce the stuck-at fault list of the 4-bit carry-save adder by collapsing equivalent faults, identify hard-to-test (untested, aborted or redundant) faults, and study the effect of pattern and coverage limits on ATPG.

Tool Used: Cadence Modus (Version 20.12-s002_1)

Fault List Reduction:

Fault list reduction eliminates equivalent and redundant faults from the circuit. This reduces the total number of faults that need to be tested, improving ATPG efficiency and reducing test time.

Commands Used:

1. build_model -designsource design.v
2. build_testmode -testmode FULLSCAN
3. verify_test_structures -testmode FULLSCAN
4. build_faultmodel
5. create_logic_tests -testmode FULLSCAN -experiment FULL
6. report_faults -testmode FULLSCAN -experiment FULL -faultstatus "untested aborted redundant" -logfile faults_hard.log
7. create_logic_tests -testmode FULLSCAN -experiment reduced_faults -append no -maxpatterns 7
8. create_logic_tests -testmode FULLSCAN -experiment 70percent -maxcoverage 70.0

Verilog Code:

```
// Gate-level 4-bit Carry-Save Adder (3:2 compressor)
// Adds three 4-bit operands X, Y, Z into a sum vector S and a carry vector C.
// Final result = S + (C << 1)
// Each gate is wrapped in its own one-gate cell so that Modus places
```

```
// stuck-at faults on every gate pin (cell-boundary fault model).

module XOR2 (input A, input B, output O);
    xor g (O, A, B);
endmodule

module AND2 (input A, input B, output O);
    and g (O, A, B);
endmodule

module OR2 (input A, input B, output O);
    or  g (O, A, B);
endmodule

module csa_4bit (
    input  [3:0] X,
    input  [3:0] Y,
    input  [3:0] Z,
    output [3:0] S,    // bitwise sum
    output [3:0] C     // bitwise carry (weight 2x)
);
    wire fa0_x1, fa0_a1, fa0_a2;
    wire fa1_x1, fa1_a1, fa1_a2;
    wire fa2_x1, fa2_a1, fa2_a2;
    wire fa3_x1, fa3_a1, fa3_a2;

    // Bit 0
    XOR2 U0_1 (.A(X[0]),   .B(Y[0]),   .O(fa0_x1));
    XOR2 U0_2 (.A(fa0_x1), .B(Z[0]),   .O(S[0]));
    AND2 U0_3 (.A(X[0]),   .B(Y[0]),   .O(fa0_a1));
    AND2 U0_4 (.A(fa0_x1), .B(Z[0]),   .O(fa0_a2));
    OR2  U0_5 (.A(fa0_a1), .B(fa0_a2), .O(C[0]));

    // Bit 1
    XOR2 U1_1 (.A(X[1]),   .B(Y[1]),   .O(fa1_x1));
    XOR2 U1_2 (.A(fa1_x1), .B(Z[1]),   .O(S[1]));
    AND2 U1_3 (.A(X[1]),   .B(Y[1]),   .O(fa1_a1));
    AND2 U1_4 (.A(fa1_x1), .B(Z[1]),   .O(fa1_a2));
    OR2  U1_5 (.A(fa1_a1), .B(fa1_a2), .O(C[1]));

    // Bit 2
    XOR2 U2_1 (.A(X[2]),   .B(Y[2]),   .O(fa2_x1));
    XOR2 U2_2 (.A(fa2_x1), .B(Z[2]),   .O(S[2]));
    AND2 U2_3 (.A(X[2]),   .B(Y[2]),   .O(fa2_a1));
    AND2 U2_4 (.A(fa2_x1), .B(Z[2]),   .O(fa2_a2));
    OR2  U2_5 (.A(fa2_a1), .B(fa2_a2), .O(C[2]));

    // Bit 3
    XOR2 U3_1 (.A(X[3]),   .B(Y[3]),   .O(fa3_x1));
    XOR2 U3_2 (.A(fa3_x1), .B(Z[3]),   .O(S[3]));
    AND2 U3_3 (.A(X[3]),   .B(Y[3]),   .O(fa3_a1));
    AND2 U3_4 (.A(fa3_x1), .B(Z[3]),   .O(fa3_a2));
    OR2  U3_5 (.A(fa3_a1), .B(fa3_a2), .O(C[3]));
endmodule
```

.BENCH Code:

```
# csa_4bit: 12 inputs, 8 outputs, 20 gates
INPUT(X0)   INPUT(X1)   INPUT(X2)   INPUT(X3)
INPUT(Y0)   INPUT(Y1)   INPUT(Y2)   INPUT(Y3)
INPUT(Z0)   INPUT(Z1)   INPUT(Z2)   INPUT(Z3)
OUTPUT(C0)  OUTPUT(C1)  OUTPUT(C2)  OUTPUT(C3)
OUTPUT(S0)  OUTPUT(S1)  OUTPUT(S2)  OUTPUT(S3)
```

```
fa0_x1 = XOR(X0, Y0)        fa1_x1 = XOR(X1, Y1)
S0 = XOR(fa0_x1, Z0)        S1 = XOR(fa1_x1, Z1)
fa0_a1 = AND(X0, Y0)        fa1_a1 = AND(X1, Y1)
fa0_a2 = AND(fa0_x1, Z0)    fa1_a2 = AND(fa1_x1, Z1)
C0 = OR(fa0_a1, fa0_a2)     C1 = OR(fa1_a1, fa1_a2)

fa2_x1 = XOR(X2, Y2)        fa3_x1 = XOR(X3, Y3)
S2 = XOR(fa2_x1, Z2)        S3 = XOR(fa3_x1, Z3)
fa2_a1 = AND(X2, Y2)        fa3_a1 = AND(X3, Y3)
fa2_a2 = AND(fa2_x1, Z2)    fa3_a2 = AND(fa3_x1, Z3)
C2 = OR(fa2_a1, fa2_a2)     C3 = OR(fa3_a1, fa3_a2)
```

(In the actual design.bench file each INPUT/OUTPUT declaration and gate is on its own line; it is shown in two columns here to save space.)

OUTPUT:

build_faultmodel – fault model generated for the design:

```
                          Global Statistics
                  ATCov  ---------------------- Global Faults -------------------------
                 Global        Total       Tested     Possibly    Redundant     Untested
Total Static       0.00          160            0            0            0          160
Collapsed Static   0.00          104            0            0            0          104
PI Static          0.00           24            0            0            0           24
PO Static          0.00           16            0            0            0           16
```

create_logic_tests -testmode FULLSCAN -experiment FULL:

```
*******************************************************************************
                          Testmode Statistics: FULLSCAN
                          #Faults  #Tested  #Possibly  #Redund  #Untested   %TCov  %ATCov
Total Static                  160      160          0        0          0  100.00  100.00

                           Global Statistics
                          #Faults  #Tested  #Possibly  #Redund  #Untested   %TCov  %ATCov
Total Static                  160      160          0        0          0  100.00  100.00
*******************************************************************************
             ----Final Pattern Statistics----
 Test Section Type                     # Test Sequences
-----------------------------------------------------------
 Logic                                        6
-----------------------------------------------------------
 Total                                        6
```

report_faults -faultstatus "untested aborted redundant" (faults_hard.log):

```
INFO (TFM-112): No faults were found which met the selection criteria based on
                faultstatus and faulttype.
INFO (TFM-901): Fault Status Statistics for faults included in this fault list:
 Total                :            0
 Tested               :            0
 Possibly Tested      :            0
 Redundant            :            0
 Untested (unprocessed):           0
```

Results before Fault reduction:

No. of Initial Faults: 160

No. of Tested Static Faults: 160

No. of Un-Tested Static Faults: 0

Fault Coverage: 100%

No. of Test Sequences: 6

Identified No. of Hard Faults: 0

Results After Fault Reduction:

No. of Faults: 104 (collapsed faults after reduction)

No. of Tested Static Faults: 104

No. of Un-Tested Static Faults: 0

Fault Coverage: 100%

No. of Test Sequences: 6

Identified No. of Hard Faults: 0

Fault collapsing reduced the fault list from 160 to 104 faults (a 35% reduction) without any loss of coverage. No untested, aborted or redundant (hard) faults remain.

reduced_faults (-maxpatterns 7): generated 6 patterns with 100% coverage. The complete test set needs only 6 patterns, so the limit of 7 was not reached.

70percent (-maxcoverage 70.0): also ended at 100% with 6 patterns. Modus generates and fault-simulates the patterns in one batch and checks the coverage limit only after the batch; for this small circuit all 6 patterns fit in the first batch. The per-pattern coverage table in Section 3 shows that coverage crosses 70% at pattern 4 (88.75%).

2. Algorithm for Test Pattern Generation

Aim: To generate test patterns and their fault-free responses for the 4-bit carry-save adder using an ATPG tool, and to determine the fault coverage they achieve.

Tool Used: Atalanta (version 2.0)

Commands Used: atalanta -t design.test design.bench

Output:

```
Welcome to Cadence tools Suite

[train@vlsi18 ece23117]$ atalanta -t design.test design.bench
before
after
end initialazition
iLastUndetectedFault=103
iLastUndetectedFault=102
iLastUndetectedFault=101
iLastUndetectedFault=99
iLastUndetectedFault=97
iLastUndetectedFault=87
iLastUndetectedFault=85
iLastUndetectedFault=83
iLastUndetectedFault=81
iLastUndetectedFault=73
iLastUndetectedFault=67
iLastUndetectedFault=61
iLastUndetectedFault=54
iLastUndetectedFault=52
iLastUndetectedFault=45
iLastUndetectedFault=44
iLastUndetectedFault=37
iLastUndetectedFault=36
iLastUndetectedFault=29
iLastUndetectedFault=28
12 12 12        ****************************************************
```

Fig. 3 – Running Atalanta on design.bench

```
12 12 12        ****************************************************
        *                                                  *
        *          Welcome to atalanta (version 2.0)       *
        *                                                  *
        *               Dong S. Ha (ha@vt.edu)             *
        *            Web: http://www.ee.vt.edu/ha          *
        *  Virginia Polytechnic Institute & State University  *
        *                                                  *
        ****************************************************

******   SUMMARY OF TEST PATTERN GENERATION RESULTS   ******
1. Circuit structure
   Name of the circuit                      : 4-bit_Carry-Save_Adder_(csa_4bit),_gate-level,_ISCAS_.bench_format
   Number of primary inputs                 : 12
   Number of primary outputs                : 8
   Number of gates                          : 20
   Level of the circuit                     : 3

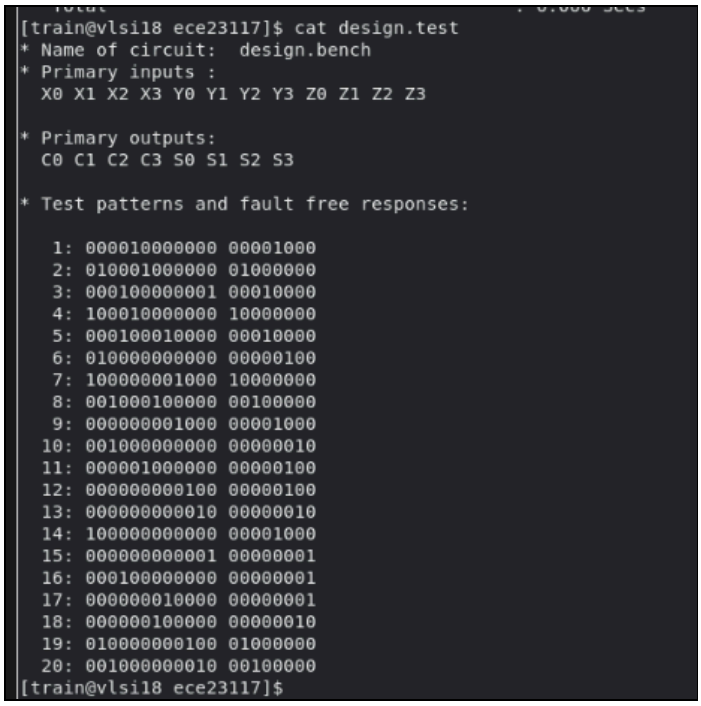
2. ATPG parameters
   Test pattern generation Mode             : RPT + DTPG + TC
   Limit of random patterns (packets)       : 16
   Backtrack limit                          : 10
   Initial random number generator seed     : 1793447561
   Test pattern compaction Mode             : REVERSE + SHUFFLE
   Limit of shuffling compaction            : 2
   Number of shuffles                       : 4

3. Test pattern generation results
   Number of test patterns before compaction : 20
   Number of test patterns after compaction  : 20
   Fault coverage                           : 100.000 %
   Number of collapsed faults               : 104
   Number of identified redundant faults    : 0
   Number of aborted faults                 : 0
   Total number of backtrackings            : 0

4. Memory used                              : 0.000 MB

5. CPU time
   Initialization                           : 0.000 Secs
   Fault simulation                         : 0.000 Secs
   FAN                                      : 0.000 Secs
   Total                                    : 0.000 Secs
```

Fig. 4 – Atalanta summary of test pattern generation results



```
[train@vlsi18 ece23117]$ cat design.test
* Name of circuit:  design.bench
* Primary inputs :
  X0 X1 X2 X3 Y0 Y1 Y2 Y3 Z0 Z1 Z2 Z3

* Primary outputs:
  C0 C1 C2 C3 S0 S1 S2 S3

* Test patterns and fault free responses:

   1: 000010000000 00001000
   2: 010001000000 01000000
   3: 000100000001 00010000
   4: 100010000000 10000000
   5: 000100010000 00010000
   6: 010000000000 00000100
   7: 100000001000 10000000
   8: 001000100000 00100000
   9: 000000001000 00001000
  10: 001000000000 00000010
  11: 000001000000 00000100
  12: 000000000100 00000100
  13: 000000000010 00000010
  14: 100000000000 00001000
  15: 000000000001 00000001
  16: 000100000000 00000001
  17: 000000010000 00000001
  18: 000000100000 00000010
  19: 010000000100 01000000
  20: 001000000010 00100000
[train@vlsi18 ece23117]$
```

Fig. 5 – Test patterns and fault-free responses (cat design.test)

Atalanta reports the first comment line of design.bench as the circuit name. Inputs are listed as X0..X3 Y0..Y3 Z0..Z3 and outputs as C0..C3 S0..S3.

Observation: Atalanta detected all 104 collapsed faults (100% fault coverage) with 20 patterns and found no redundant or aborted faults. Each pattern exercises only one full adder (the inputs of the other three are 0), and every full adder receives the same five input combinations (a b cin = 001, 010, 100, 101, 110), giving 5 × 4 = 20 patterns. Since the four full adders share no signals, the same five combinations could be applied to all bits at once, so 5 patterns would be enough for the whole circuit; Atalanta's compaction did not merge them because unspecified inputs were filled with 0 before compaction.

3. Algorithm for Fault Simulation

Aim: To perform fault simulation on a digital design in order to evaluate the effectiveness of generated test patterns in detecting faults, analyze fault coverage, and identify undetected faults for improving test quality.

Tool Used: Cadence Modus

Commands Used:

1. build_model -designsource design.v
2. build_testmode -testmode FULLSCAN
3. verify_test_structures -testmode FULLSCAN
4. build_faultmodel
5. create_logic_tests -testmode FULLSCAN -experiment FULL
6. write_vectors -testmode FULLSCAN -inexperiment FULL -language verilog
7. report_fault_statistics -testmode FULLSCAN -experiment FULL
8. write_vectors -testmode FULLSCAN -inexperiment FULL -language stil
9. create_logic_tests -testmode FULLSCAN -experiment C70 -maxcoverage 70.0
10. report_fault_statistics -testmode FULLSCAN -experiment C70
11. create_logic_tests -testmode FULLSCAN -experiment HS -maxpatterns 1
12. create_logic_tests -testmode FULLSCAN -experiment GP -maxpatterns 3
13. create_logic_tests -testmode FULLSCAN -experiment MP -maxpatterns 5

verify_test_structures reported 0 scan chains and no rule violations: the design is purely combinational (no flip-flops), so it is tested directly through its 12 primary inputs and 8 primary outputs.

OUTPUT:

VER.FULLSCAN.FULL.data.logic.ex1.ts1.verilog (generated by write_vectors):

```
203 Z                          151 4 0 0
104 MODEINIT                   900 1.1.1.2.4.1
104                            901 1.1.1.2.4.2
151 1 0 0                      900 1.1.1.2.4.2
900 1.1.1.2.1.1                200 101001000001
901 1.1.1.2.1.2                202 00001111
900 1.1.1.2.1.2                400
200 110011000000               151 5 0 0
202 11000000                   900 1.1.1.2.5.1
400                            901 1.1.1.2.5.2
151 2 0 0                      900 1.1.1.2.5.2
900 1.1.1.2.2.1                200 110000111000
901 1.1.1.2.2.2                202 10000111
900 1.1.1.2.2.2                400
200 101011111101               151 6 0 0
202 11111000                   900 1.1.1.2.6.1
400                            901 1.1.1.2.6.2
151 3 0 0                      900 1.1.1.2.6.2
900 1.1.1.2.3.1                200 011100001110
901 1.1.1.2.3.2                202 01101001
900 1.1.1.2.3.2                400
```

```
200 000110010110                200 011100001110
202 00011110                    400
400
```

Key: 203 Z – all inputs at high impedance; 104 MODEINIT – test-mode initialisation; 151 n – start of pattern n; 900/901 – stimulus and measure events; 200 – 12 input values (X0..X3, Y0..Y3, Z0..Z3); 202 – 8 expected outputs (C0..C3, S0..S3); 400 – end of pattern. The final 200/400 pair repeats pattern 6 to hold the inputs after the last test; it is not a seventh test.

Decoded FULL test patterns (every response verified against the design):

#	X0..X3	Y0..Y3	Z0..Z3	C0..C3	S0..S3	FA0..FA3 inputs (a b cin)
1	1100	1100	0000	1100	0000	110, 110, 000, 000
2	1010	1111	1101	1111	1000	111, 011, 110, 011
3	0001	1001	0110	0001	1110	010, 001, 001, 110
4	1010	0100	0001	0000	1111	100, 010, 100, 001
5	1100	0011	1000	1000	0111	101, 100, 010, 010
6	0111	0000	1110	0110	1001	001, 101, 101, 100

Cumulative coverage after each FULL pattern (write_vectors coverage estimate):

Pattern	1	2	3	4	5	6
Static coverage (%)	2.50	31.25	44.38	88.75	96.25	100.00
Delta (%)	2.50	28.75	13.12	44.38	7.50	3.75

report_fault_statistics -testmode FULLSCAN -experiment FULL:

```
                ---- ATCov ---- ---------------- Testmode Faults ----------------
                Testmode Global   Total  Tested  Possibly  Redundant  Untested
Total Static      100.00 100.00     160     160         0          0         0
Collapsed Static  100.00 100.00     104     104         0          0         0
PI Static         100.00 100.00      24      24         0          0         0
PO Static         100.00 100.00      16      16         0          0         0
```

create_logic_tests -testmode FULLSCAN -experiment C70 -maxcoverage 70.0: finished at 100% coverage with 6 test sequences, identical to FULL, because the coverage limit is checked only after the whole batch of 6 patterns has been simulated.

create_logic_tests -testmode FULLSCAN -experiment HS -maxpatterns 1 :

```
*******************************************************************************
                          Testmode Statistics: FULLSCAN
                          #Faults  #Tested  #Possibly  #Redund  #Untested   %TCov  %ATCov
Total Static                  160       64          0        0         96   40.00   40.00
                           Global Statistics
Total Static                  160       64          0        0         96   40.00   40.00
*******************************************************************************
             ----Final Pattern Statistics----
 Logic                                    1
 Total                                    1
```

When only 1 test pattern is used, the fault coverage is limited to 40.00%. A large number of faults (96) remain undetected, because one pattern cannot exercise both the sum and carry paths of all four full adders.

create_logic_tests -testmode FULLSCAN -experiment GP -maxpatterns 3 :

```
*******************************************************************************
                          Testmode Statistics: FULLSCAN
                          #Faults  #Tested  #Possibly  #Redund  #Untested   %TCov  %ATCov
Total Static                  160      134          0        0         26   83.75   83.75
                           Global Statistics
Total Static                  160      134          0        0         26   83.75   83.75
*******************************************************************************
             ----Final Pattern Statistics----
 Logic                                    3
 Total                                    3
```

Increasing the number of patterns to 3 improves fault coverage to 83.75%. The number of untested faults reduces significantly (to 26), demonstrating improved detection efficiency.

create_logic_tests -testmode FULLSCAN -experiment MP -maxpatterns 5 :

```
*******************************************************************************
                          Testmode Statistics: FULLSCAN
                          #Faults  #Tested  #Possibly  #Redund  #Untested   %TCov  %ATCov
Total Static                  160      154          0        0          6   96.25   96.25
                           Global Statistics
Total Static                  160      154          0        0          6   96.25   96.25
*******************************************************************************
             ----Final Pattern Statistics----
 Logic                                    5
 Total                                    5
```

With 5 test patterns, fault coverage increases further to 96.25% and approaches complete fault detection; only 6 faults remain, which the sixth pattern of the FULL experiment detects. This shows that increasing test patterns enhances fault coverage.

Experiment	Patterns	Coverage (%)	Time (s)	Memory (MB)	Untested Faults	Efficiency
HS	1	40.00	3.36	6.264	96	11.90
GP	3	83.75	2.34	6.268	26	35.79
MP	5	96.25	3.20	6.272	6	30.08
FULL	6	100.00	2.36	6.239	0	42.37

Efficiency = Coverage (%) ÷ Elapsed time (s). Time and memory are taken from the TDA-001 summary of each create_logic_tests run.

Observation

As the number of test patterns increases, fault coverage improves (40.00% → 83.75% → 96.25% → 100%) while the number of untested faults decreases (96 → 26 → 6 → 0). This demonstrates the trade-off between test cost (number of patterns) and fault coverage.

Comparison of Atalanta and Cadence Modus

Quantity	Atalanta	Cadence Modus
Collapsed faults	104	104 (from 160 total)
Fault coverage	100%	100%
Redundant / aborted faults	0 / 0	0 / 0
Test patterns	20	6
Theoretical minimum	5	5

Conclusion

Fault list reduction cut the fault list by 35% (160 → 104 faults) without any loss of coverage, and no hard faults were found. Both tools detected all 104 collapsed stuck-at faults. Cadence Modus produced a compact 6-pattern test set, close to the 5-pattern minimum, while Atalanta needed 20 patterns, showing the effect of test compaction on test length. The HS, GP and MP experiments confirm that fault coverage rises with the number of test patterns.